Etude de l'influence des boucles de rétroactions négatives sur les performance d'un robot

Samuel FERNANDES

Introduction

Asservissement

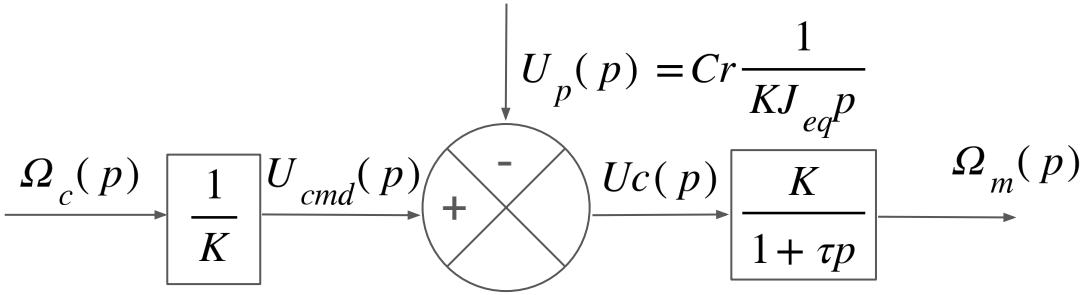


Figure 1 : Schéma bloc en boucle ouverte

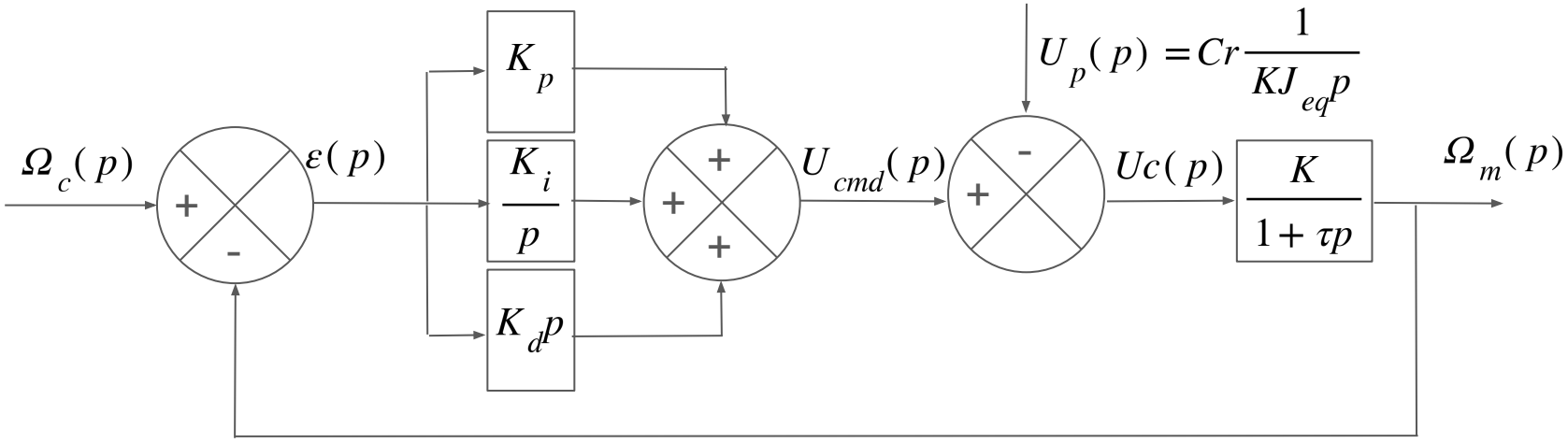


Figure 2 : Schéma bloc en boucle fermée avec PID

Problématique

Comment l'utilisation de cycles de mesure et de correction dans une boucle de rétroaction permet-elle d'améliorer la précision et la stabilité du mouvement d'un robot ?

Sommaire

I. **Présentation du système**

- ***Robot et ses composants***
- ***Chaîne de commande et acquisition des données***

II. **Etude théorique**

- ***Modèles du moteur et de l'environnement simulé***
- ***Régulation PID et critères de performance***

III. **Résultats et analyse**

- ***Simulation***
- ***Expérimentations réelles***

I. Présentation du système

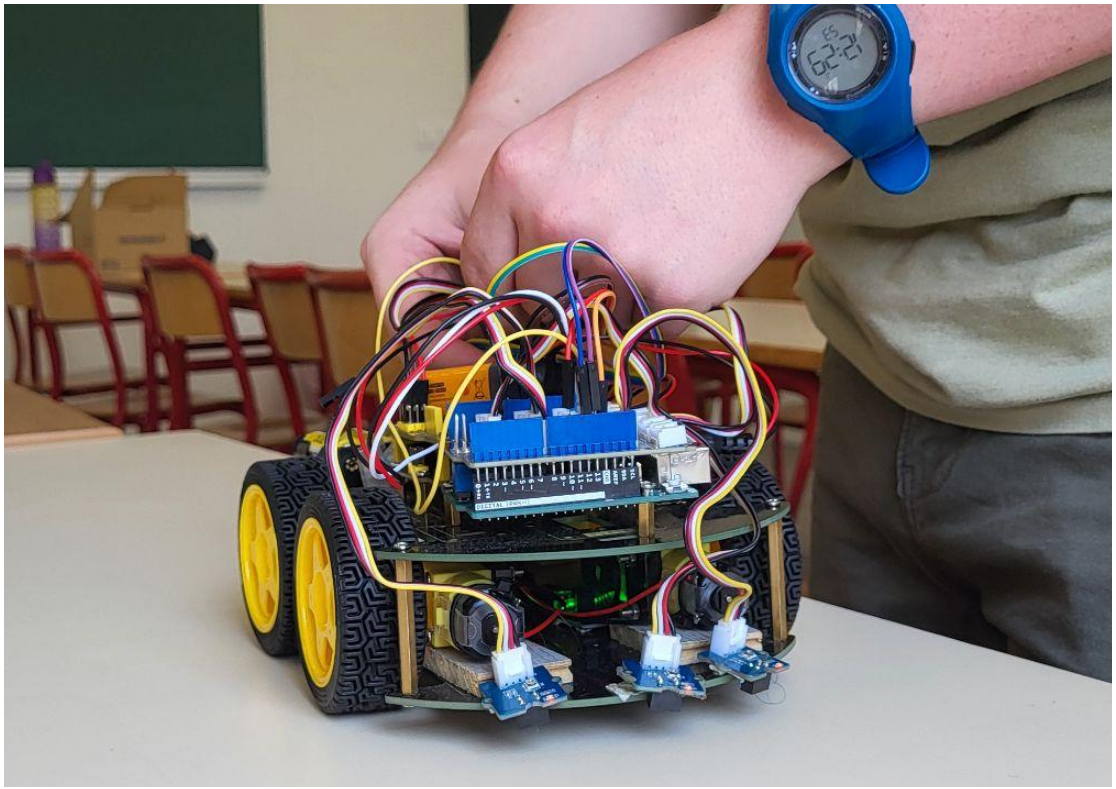


Figure 3 : photo du robot

Composants :

- **4 moteurs**
- **3 capteurs suiveurs de lignes**
- **2 encodeurs +**
- **2 microcontrôleurs**
- **6 piles 1,5 V**
- **1 pile 9V**
- **bouton poussoir**
- **1 module bluetooth**

I. Présentation du système

Fonctionnement du système

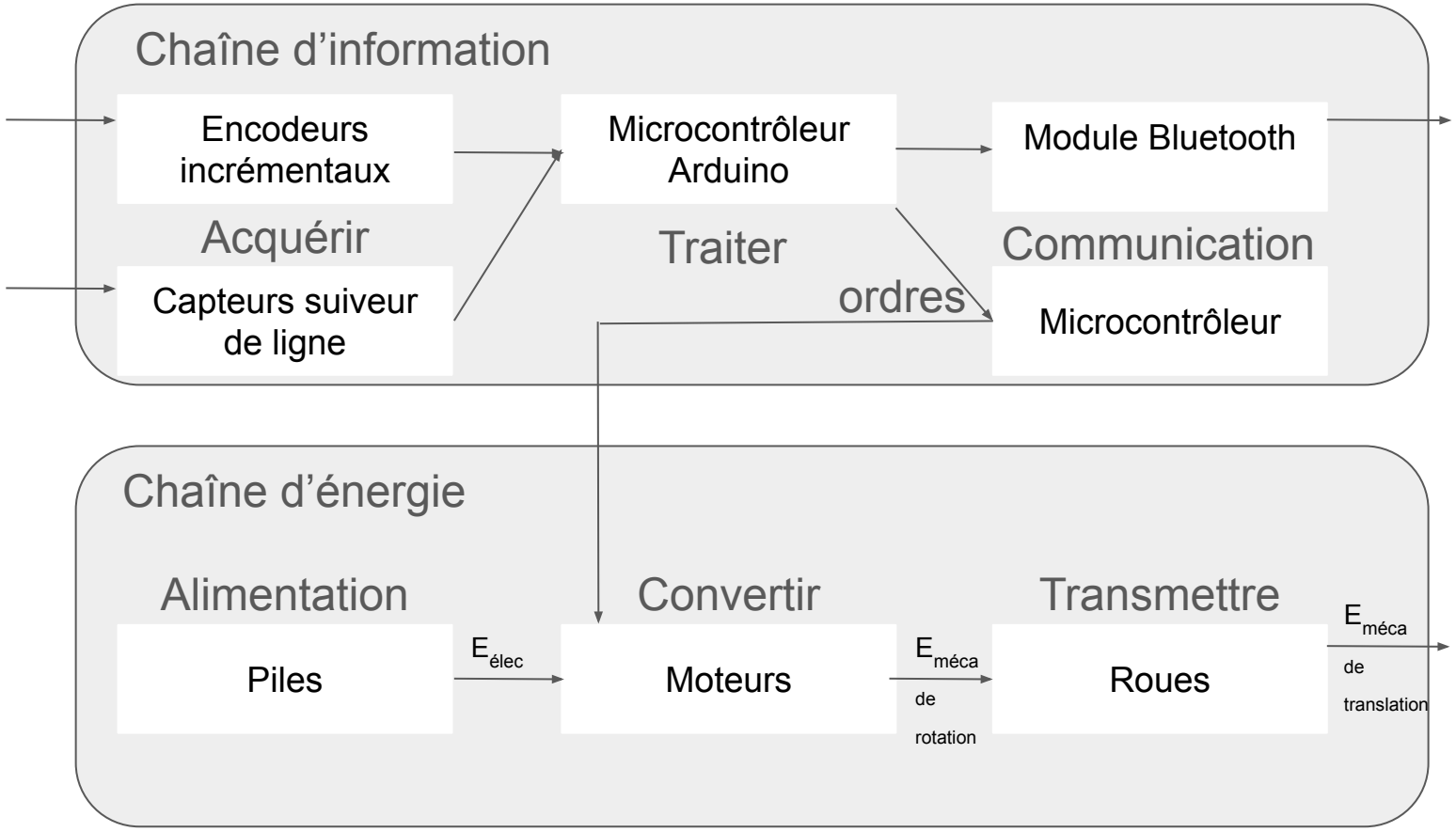


Figure 4 : Schéma topo fonctionnel du système

II. Etude théorique

Modèle moteur

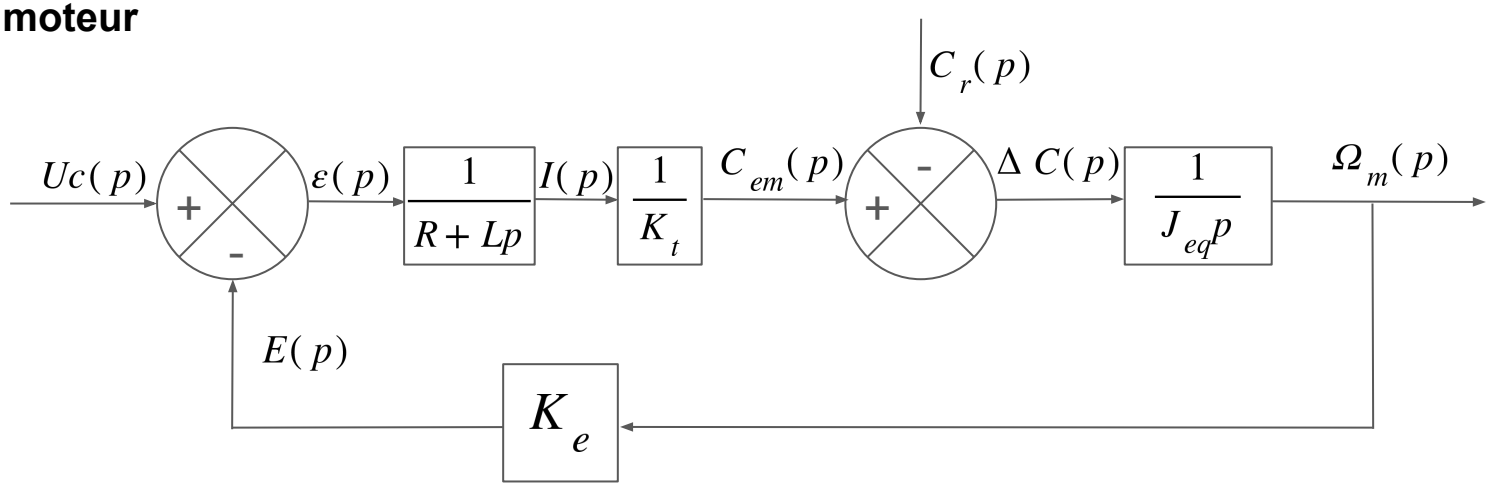


Figure 5 : Schéma bloc d'un moteur

$$H(p) = \frac{1/K_e}{1 + \left(RJ_{eq}/K_tK_e\right)p + \left(LJ_{eq}/K_tK_e\right)p^2}$$

hypothèse de travail : $H(p) = \frac{K}{(1+\tau p)}$

$$\frac{d\omega_m}{dt} + \frac{1}{\tau}\omega_m = \frac{K}{\tau}U_c$$

```
reponse_g += (K_g * commande_g + perturbations - reponse_g) * (dt / tau_g)
reponse_d += (K_d * commande_d + perturbations - reponse_d) * (dt / tau_d)
```

Figure 6 : programme de calcul de la réponse par la méthode d'Euler

II. Etude théorique

Simulation des perturbations

```
# Mise à jour de la consigne en fonction de la position du robot
if x % (4 + np.pi) > 1:
    consigne_g = 8.11 # val en rad/s
if x % (4 + np.pi) > (1 + 0.5 * np.pi):
    consigne_g = 12.56 # val en rad/s
if x % (4 + np.pi) > (3 + 0.5 * np.pi):
    consigne_g = 8.11 # val en rad/s
if x % (4 + np.pi) > (3 + np.pi):
    consigne_g = 12.56 # val en rad/s

# Ajout de perturbations aléatoires et événements ponctuels
perturbations = (0.2 * np.random.normal() +          # bruit aléatoire
                 (-2 if 1.25 + 0.5 * np.pi < x % (4 + np.pi) < 1.75 + 0.5 * np.pi  else 0) +
                 (2 if 2.25 + 0.5 * np.pi < x % (4 + np.pi) < 2.75 + 0.5 * np.pi  else 0) + # événement ponctuel
                 0.1 * np.sin(2 * np.pi * 0.3 * e)  # environnement irrégulier
                 )
```

Figure 7 : Programme de changement de consigne et de génération de perturbations en fonction de la position

II. Etude théorique

Régulation PID

P : Corrige proportionnellement à l'erreur instantanée

I: Corrige l'erreur cumulée dans le temps

D: Anticipe les variations de l'erreur, amortit les oscillations

IAE : Mesure l'écart total entre la consigne et la réponse, sans tenir compte du signe.

ISE : Pénalise fortement les grandes erreurs.

ITAE : Pénalise les erreurs qui durent longtemps.

ITSE : Pénalise encore plus les grandes erreurs persistantes.

$$IAE = \int_0^\infty |\varepsilon(t)|\,dt$$

$$ISE = \int_0^\infty \varepsilon(t)^2\,dt$$

$$ITAE = \int_0^\infty t|\varepsilon(t)|\,dt$$

$$ITSE = \int_0^\infty t\varepsilon(t)^2\,dt$$

```
# Critères d'évaluation :
ISE = np.trapezoid(erreur_r**2, t)           # Integral of Squared Error
IAE = np.trapezoid(np.abs(erreur_r), t)      # Integral of Absolute Error
ITSE = np.trapezoid(t * erreur_r**2, t)      # Integral of Time * Squared Error
ITAE = np.trapezoid(t * np.abs(erreur_r), t) # Integral of Time * Absolute Error
```

Figure 8 : Programme de calcul des critères de performance

III. Résultats et analyse

Résultats de simulation

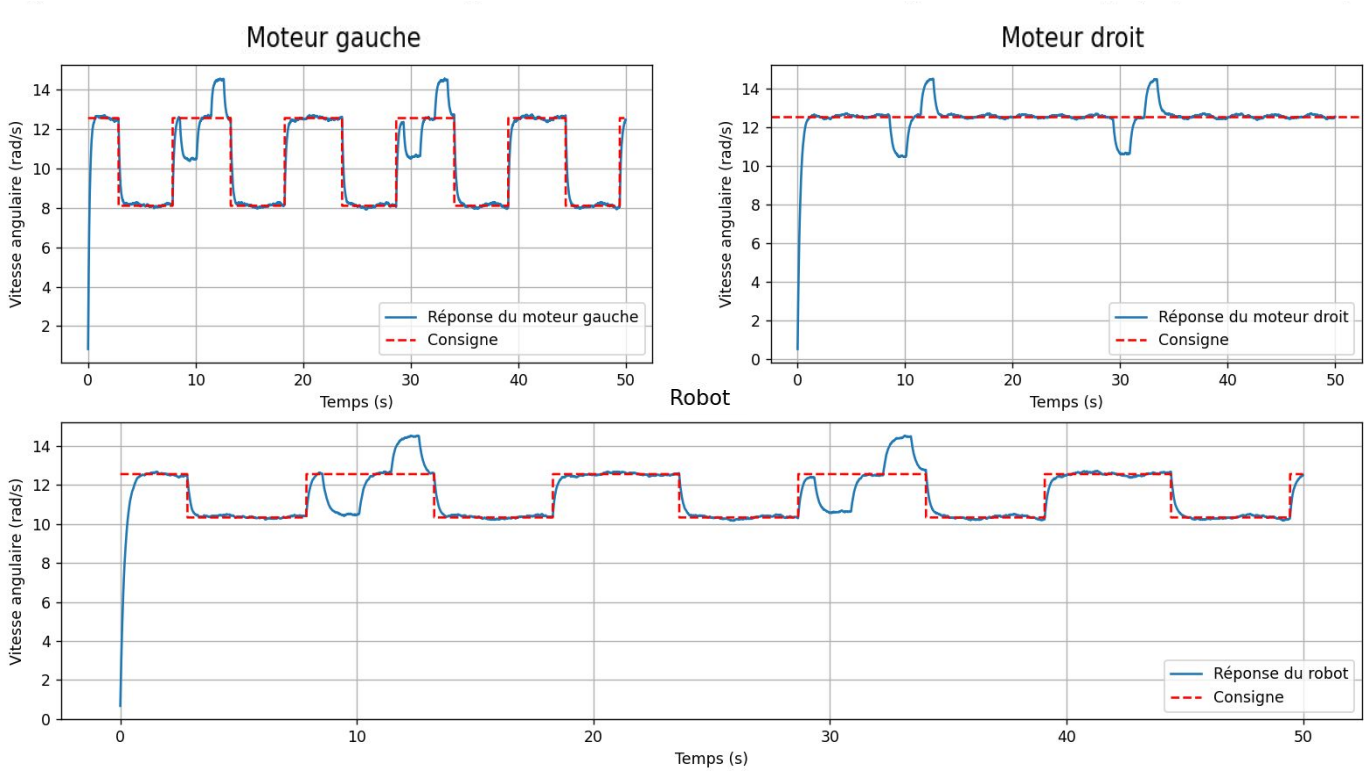


Figure 9 : Boucle ouverte - Réponse des moteurs et du robot sans régulation

III. Résultats et analyse

Résultats de simulation

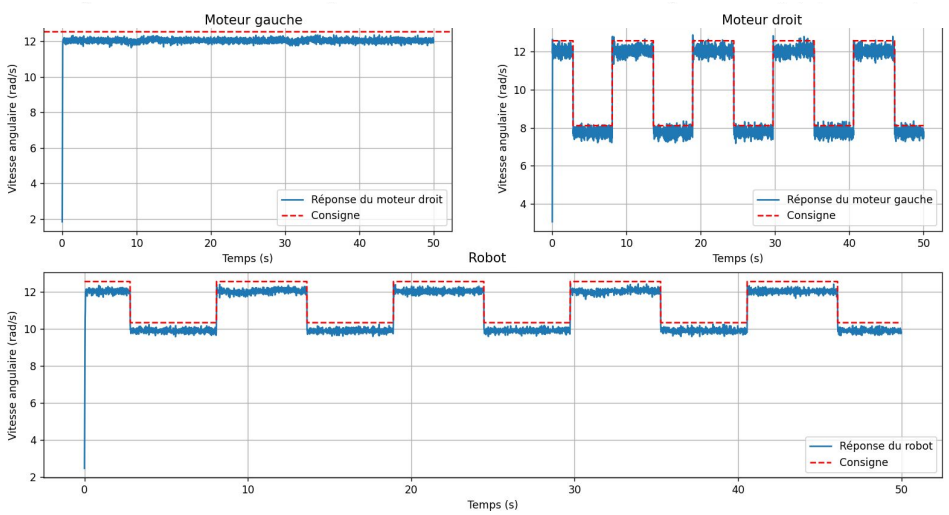


Figure 10 : Boucle fermée - Réponse des moteurs et du robot avec régulation Proportionnelle

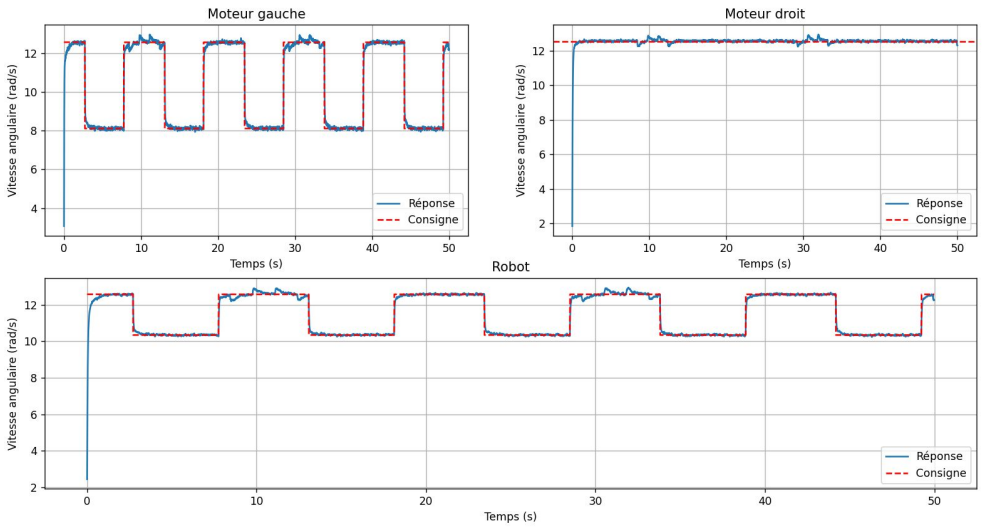


Figure 11 : Boucle fermée - Réponse des moteurs et du robot avec régulation PID (2, 6, 0.005116)

III. Résultats et analyse

Résultats de simulation

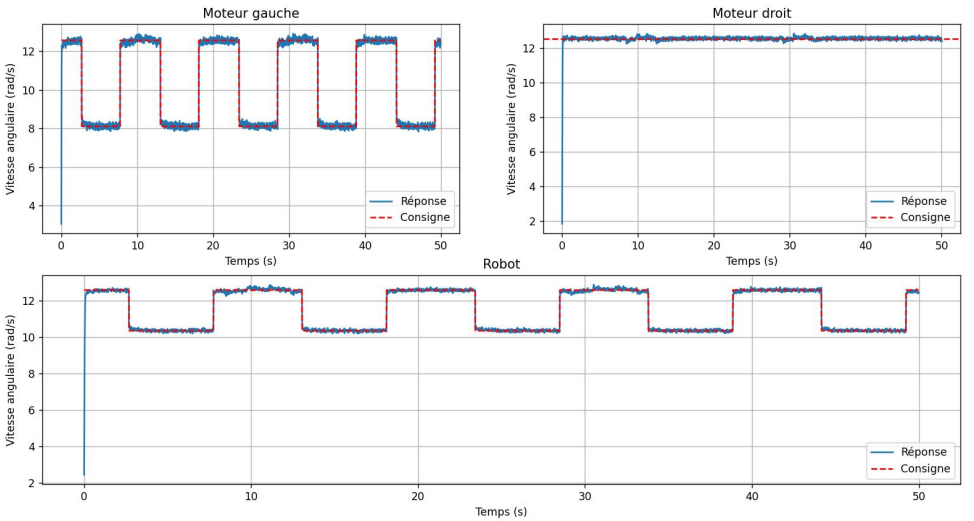


Figure 12 : Boucle fermée - Réponse des moteurs et du robot avec régulation PID (4.968327, 9.966255, 0.005116)

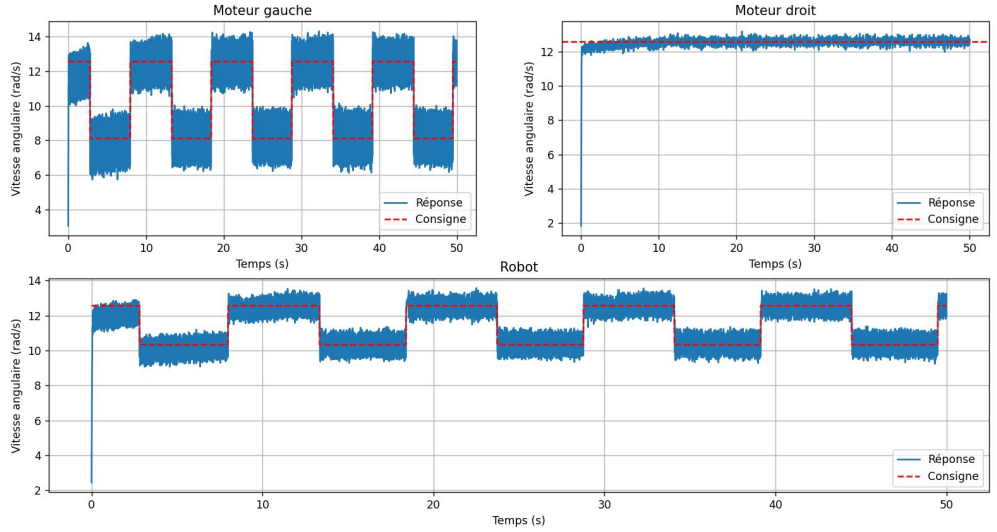


Figure 13 : Boucle fermée - Réponse des moteurs et du robot avec régulation PID (9,2,0.04)

III. Résultats et analyse

Résultats de simulation

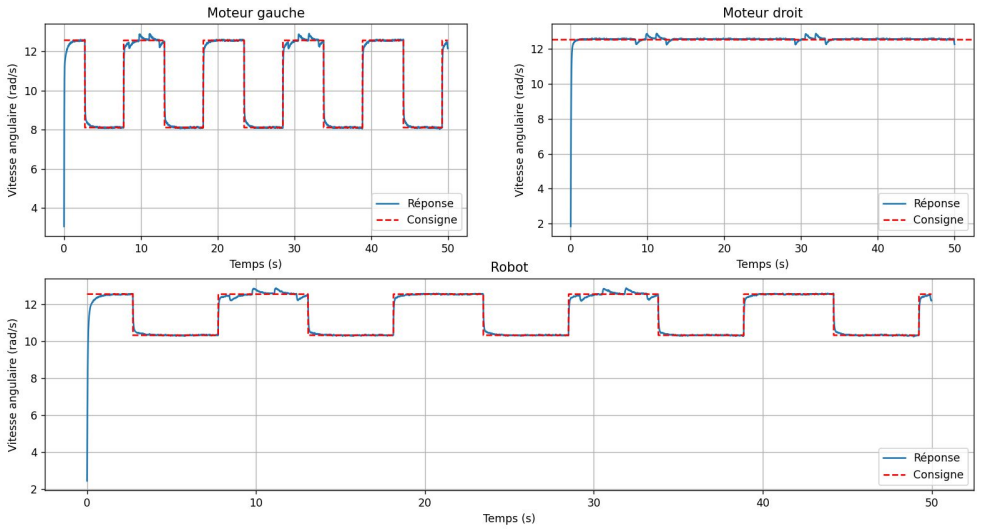


Figure 14 : Boucle fermée - Réponse des moteurs et du robot avec régulation PID sans bruit (2, 6, 0.005116)

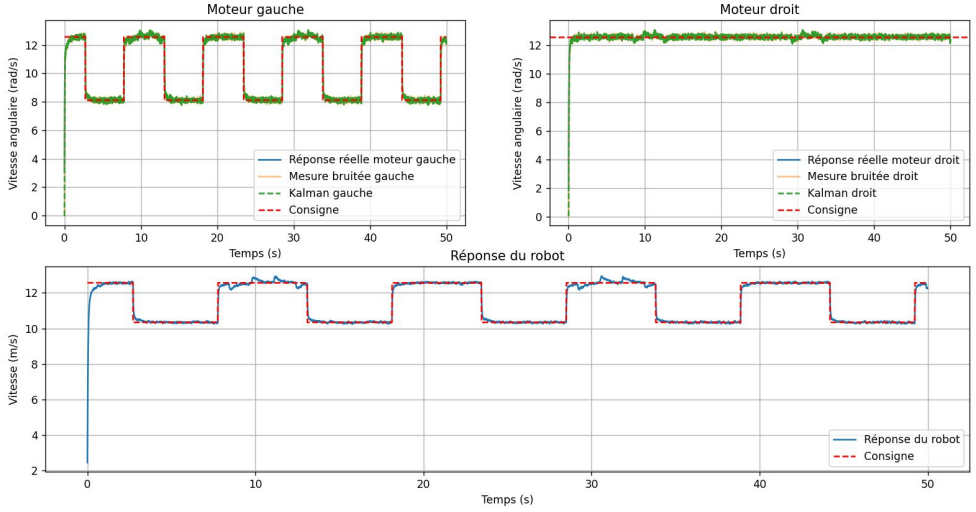


Figure 15 : Boucle fermée - Réponse des moteurs et du robot avec régulation PID avec filtre Kalman 1d (2, 6, 0.005116)

III. Résultats et analyse

Expérimentations réelles

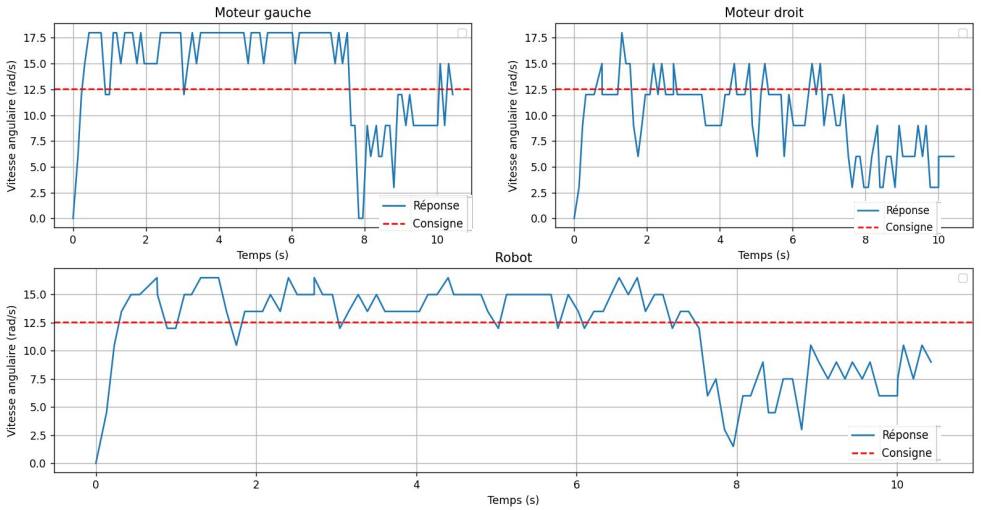


Figure 16 : Boucle ouverte - Réponse réelle des moteurs et du robot sans régulation

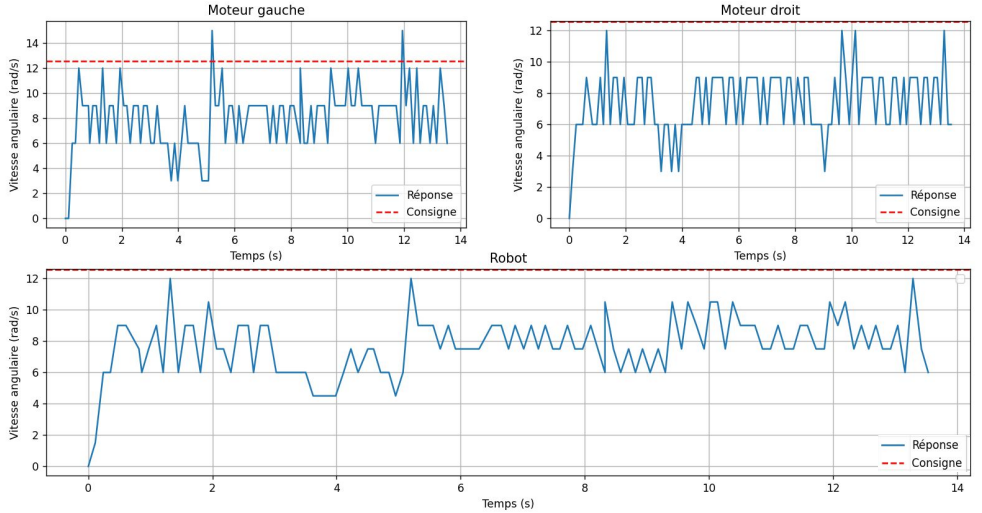


Figure 17 : Boucle fermée - Réponse réelle des moteurs et du robot avec régulation PID (2, 6, 0.005116)

Conclusion

- **Bilan**
- **Limites**
- **Perspectives**

Annexes

Annexe 1 : Programme python simulation boucle ouverte

```python
import numpy as np
import matplotlib.pyplot as plt
import math

# Paramètres système
K_g = 2.3       # gain statique
tau_g = 0.15     # constante de temps
K_d = 2.3      # gain statique
tau_d = 0.25     # constante de temps
dt = 0.01     # pas de temps
t_max = 50   # durée de la simulation

# Consigne
consigne_g = 12.56 #val en rad/s
consigne_d = 12.56 #val en rad/s
perturbations=0
t = np.arange(0, t_max, dt)
reponse_g = 0
reponse_d = 0
sortie_g = []
sortie_d = []
sortie_r = []
X=[]
Cg=[]
Cr=[]
x=0

for e in t:
    # Stocke la consigne du moteur gauche pour la courbe et calcule la consigne du robot comme moyenne des deux moteurs
    Cg.append(consigne_g)
    Cr.append(0.5 * (consigne_g + consigne_d))

    # Mise à jour de la réponse dynamique du moteur gauche (filtre du premier ordre)
    reponse_g += (consigne_g + perturbations - reponse_g) * (dt / tau_g)
    # Mise à jour de la réponse dynamique du moteur droit (filtre du premier ordre)
    reponse_d += (consigne_d + perturbations - reponse_d) * (dt / tau_d)

    # Conversion des réponses dynamiques en vitesses linéaires à l'aide d'un coefficient de conversion
    vg = 0.03 * reponse_g
    vd = 0.03 * reponse_d

    # Calcul de la vitesse du robot comme la moyenne des vitesses des deux moteurs
    vr = (vg + vd) * 0.5

    # Intégration de la vitesse pour obtenir la position x
    x += vr * dt

    # Sauvegarde des données pour affichage
    X.append(x)
    sortie_g.append(reponse_g)
    sortie_d.append(reponse_d)
    sortie_r.append(0.5 * (reponse_g + reponse_d))

    # Mise à jour de la consigne en fonction de la position x modifiée
    # (Utilisation du modulo pour définir différents intervalles de consigne)
    if x % (4 + math.pi) > 1:
        consigne_g = 8.11
    if x % (4 + math.pi) > (1 + 0.5 * math.pi):
        consigne_g = 12.56
    if x % (4 + math.pi) > (3 + 0.5 * math.pi):
        consigne_g = 8.11
    if x % (4 + math.pi) > (3 + math.pi):
        consigne_g = 12.56

    # Calcul des perturbations pour simuler un environnement irrégulier :
    perturbations = (0.2 * np.random.normal() +           # bruit aléatoire
                     (-2 if 1.25 + 0.5 * np.pi < x % (4 + np.pi) < 1.75 + 0.5 * np.pi else 0) +
                     (2 if 2.25 + 0.5 * np.pi < x % (4 + np.pi) < 2.75 + 0.5 * np.pi else 0) + # événement ponctuel
                     0.1 * np.sin(2 * np.pi * 0.3 * e)  # composante sinusoïdale pour perturbation périodique
                    )



# Affichage
plt.figure(figsize=(20, 3))
plt.subplot(2,2,1)
plt.plot(t, sortie_g, label="Réponse du moteur gauche")
plt.plot(t, Cg, color='r', linestyle='--', label="Consigne", )
plt.xlabel("Temps (s)")
plt.ylabel("Vitesse angulaire (rad/s)")
plt.grid(True)
plt.legend()
plt.subplot(2,2,2)
plt.plot(t, sortie_d, label="Réponse du moteur droit")
plt.axhline(y=12.56, color='r', linestyle='--', label="Consigne")
plt.xlabel("Temps (s)")
plt.ylabel("Vitesse angulaire (rad/s)")
plt.grid(True)
plt.legend()
plt.subplot(2,1,2)
plt.plot(t, sortie_r, label="Réponse du robot")
plt.plot(t, Cr, color='r', linestyle='--', label="Consigne")
plt.xlabel("Temps (s)")
plt.ylabel("Vitesse angulaire (rad/s)")
plt.grid(True)
plt.legend()

plt.subplots_adjust(top=0.9)
plt.suptitle("Figure 9 : Boucle ouverte - Réponse des moteurs et du robot sans régulation")
plt.show()
```

Annexes

Annexe 2 : Programme python simulation boucle fermée simple

```python
import numpy as np
import matplotlib.pyplot as plt
import math

# Paramètres système
K_g = 2.3       # gain statique du moteur gauche
tau_g = 0.15     # constante de temps du moteur gauche
K_d = 2.3      # gain statique du moteur droit
tau_d = 0.25     # constante de temps du moteur droit
dt = 0.01     # pas de temps
t_max = 50   # durée de la simulation

# PID (proportionnel seulement)
Kp_g, Kp_d = 10, 10

# Temps et initialisation
consigne_g = 12.56 #val en rad/s
consigne_d = 12.56 #val en rad/s
perturbations=0
t = np.arange(0, t_max, dt)
reponse_g = 0
reponse_d = 0
sortie_g = []
sortie_d = []
sortie_r = []
X=[]
Cg=[]
Cr=[] #np.vect
x=0

for e in t:
    # Stocker la consigne du moteur gauche et la consigne moyenne pour le robot
    Cg.append(consigne_g)
    Cr.append(0.5 * (consigne_g + consigne_d))

    # Calcul des erreurs avec ajout de bruit sur la mesure
    erreur_g = consigne_g - reponse_g + np.random.normal(0, 0.1)  # Erreur pour le moteur gauche
    erreur_d = consigne_d - reponse_d + np.random.normal(0, 0.1)  # Erreur pour le moteur droit

    # Calcul des commandes en utilisant la régulation proportionnelle
    commande_g = Kp_g * erreur_g  # Commande pour le moteur gauche
    commande_d = Kp_d * erreur_d  # Commande pour le moteur droit

    # Contrainte des commandes à la plage [-20, 20]
    commande_g = np.clip(commande_g, -20, 20)
    commande_d = np.clip(commande_d, -20, 20)

    # Mise à jour de la réponse des moteurs selon un modèle dynamique de premier ordre
    reponse_g += (K_g * commande_g + perturbations - reponse_g) * (dt / tau_g)
    reponse_d += (K_d * commande_d + perturbations - reponse_d) * (dt / tau_d)

    # Calcul des vitesses issues des réponses des moteurs
    vg = 0.03 * reponse_g  # Vitesse du moteur gauche
    vd = 0.03 * reponse_d  # Vitesse du moteur droit
    vr = 0.5 * (vg + vd)   # Vitesse du robot (moyenne des deux moteurs)

    # Mise à jour de la position du robot par intégration
    x += vr * dt

    # Stockage des valeurs pour tracé ultérieur
    X.append(x)
    sortie_g.append(reponse_g)
    sortie_d.append(reponse_d)
    sortie_r.append(0.5 * (reponse_g + reponse_d))

    # Modification de la consigne du moteur gauche en fonction de la position du robot
    if x % (4 + math.pi) > 1:
        consigne_g = 8.11
    if x % (4 + math.pi) > (1 + 0.5 * math.pi):
        consigne_g = 12.56
    if x % (4 + math.pi) > (3 + 0.5 * math.pi):
        consigne_g = 8.11
    if x % (4 + math.pi) > (3 + math.pi):
        consigne_g = 12.56

    # Calcul des perturbations incluant :
    perturbations = (
        0.2 * np.random.normal() +
        (-2 if 1.25 + 0.5 * np.pi < x % (4 + np.pi) < 1.75 + 0.5 * np.pi else 0) +
        (2 if 2.25 + 0.5 * np.pi < x % (4 + np.pi) < 2.75 + 0.5 * np.pi else 0) +
        0.1 * np.sin(2 * np.pi * 0.3 * e)
    )

# Affichage
plt.figure(figsize=(20, 3))
plt.subplot(2, 2,2)
plt.plot(t, sortie_g, label="Réponse du moteur gauche")
plt.plot(t, Cg, color='r', linestyle='--', label="Consigne")
plt.xlabel("Temps (s)")
plt.ylabel("Vitesse angulaire (rad/s)")
plt.grid(True)
plt.legend()
plt.subplot(2, 2,1)
plt.plot(t, sortie_d, label="Réponse du moteur droit")
plt.axhline(y=12.56, color='r', linestyle='--', label="Consigne")
plt.xlabel("Temps (s)")
plt.ylabel("Vitesse angulaire (rad/s)")
plt.grid(True)
plt.legend()
plt.subplot(2, 1,2)
plt.plot(t, sortie_r, label="Réponse du robot")
plt.plot(t, Cr, color='r', linestyle='--', label="Consigne")
plt.xlabel("Temps (s)")
plt.ylabel("Vitesse angulaire (rad/s)")
plt.grid(True)
plt.legend()

plt.subplots_adjust(top=0.9)
plt.suptitle("Figure 10 : Boucle fermée - Réponse des moteurs et du robot avec régulation Proportionnelle")
plt.show()
```

Annexes

Annexe 3 : Programme python simulation boucle fermée avec PID

```python
import numpy as np
import matplotlib.pyplot as plt

K_g, K_d = 2.3, 2.3        # gains statiques des moteurs gauche et droit
tau_g, tau_d = 0.15, 0.25  # constantes de temps des moteurs gauche et droit
dt = 0.01                  # pas de temps
t_max = 50                 # durée de la simulation

Kp_g,  Ki_g, Kd_g, Kp_d,  Ki_d, Kd_d = 2, 6, 0.005116, 2, 6, 0.005116  # coefficients du PID
#9,2,0.04     k/tau base
#4.968327, 9.966255, 0.005116    k/tau robot
# 2, 6, 0.005116, 2, 6, 0.005116
consigne_g = consigne_d = 12.56637061         # initialisation des consignes de vitesse angulaire en rad/s
perturbations = 0                             # initialisation des perturbations

t = np.arange(0, t_max, dt)
reponse_g = reponse_d = 0 # initialisation des réponses des moteurs gauche et droit

# initialisation des listes pour stocker les réponses et les consignes
sortie_g = []
sortie_d = []
sortie_r = []

X = []
Cg = []
Cr = [] # consignes du robot
x = 0 # position du robot

integrale_g = integrale_d = 0 # initialisation des intégrales pour le PID
erreur_precedente_g = erreur_precedente_d = 0



for e in t:
    # Stockage des consignes pour le moteur gauche et la commande moyenne pour le robot
    Cg.append(consigne_g)
    Cr.append(0.5 * (consigne_g + consigne_d))

    # Calcul des erreurs pour les moteurs gauche et droit (avec ajout de bruit nul)
    erreur_g = consigne_g - reponse_g + np.random.normal(0, 0.1)
    erreur_d = consigne_d - reponse_d + np.random.normal(0, 0.1)

    # Mise à jour de l'intégrale des erreurs pour le PID
    integrale_g += erreur_g * dt
    integrale_d += erreur_d * dt

    # Limitation de l'intégrale pour éviter l'accumulation excessive (anti-windup)
    integrale_g = np.clip(integrale_g, -10, 10)
    integrale_d = np.clip(integrale_d, -10, 10)
```

```python
    # Calcul de la dérivée des erreurs pour le PID
    derivee_g = (erreur_g - erreur_precedente_g) / dt
    derivee_d = (erreur_d - erreur_precedente_d) / dt

    # Mise à jour des erreurs précédentes pour l'itération suivante
    erreur_precedente_g = erreur_g
    erreur_precedente_d = erreur_d

    # Calcul de la commande PID pour chaque moteur
    commande_g = Kp_g * erreur_g + Ki_g * integrale_g + Kd_g * derivee_g
    commande_d = Kp_d * erreur_d + Ki_d * integrale_d + Kd_d * derivee_d

    # Limitation de la commande pour rester dans des valeurs acceptables
    commande_g = np.clip(commande_g, -20, 20)
    commande_d = np.clip(commande_d, -20, 20)

    # Mise à jour de la réponse du système pour chaque moteur (modélisation du comportement dynamique)
    reponse_g += (K_g * commande_g + perturbations - reponse_g) * (dt / tau_g)
    reponse_d += (K_d * commande_d + perturbations - reponse_d) * (dt / tau_d)

    # Calcul de la vitesse en m/s pour chaque moteur et pour le robot
    vg = 0.03 * reponse_g  # vitesse du moteur gauche
    vd = 0.03 * reponse_d  # vitesse du moteur droit
    vr = (vg + vd) * 0.5   # vitesse moyenne du robot
    x += vr * dt           # mise à jour de la position du robot (intégration de la vitesse)

    # Stockage de la position et des réponses pour analyse ultérieure
    X.append(x)
    sortie_g.append(reponse_g)
    sortie_d.append(reponse_d)
    sortie_r.append(0.5 * (reponse_g + reponse_d))

    # Ajustement de la consigne en fonction de la position du robot selon un cycle périodique
    if x % (4 + np.pi) > 1:
        consigne_g = 8.11  # consigne inférieure en rad/s
    if x % (4 + np.pi) > (1 + 0.5 * np.pi):
        consigne_g = 12.56 # consigne supérieure en rad/s
    if x % (4 + np.pi) > (3 + 0.5 * np.pi):
        consigne_g = 8.11  # repasse à la consigne inférieure
    if x % (4 + np.pi) > (3 + np.pi):
        consigne_g = 12.56 # consigne supérieure en rad/s

    # Ajout de perturbations aléatoires et événements ponctuels
    perturbations = (0.2 * np.random.normal() +                          # bruit aléatoire
                     (-2 if 1.25 + 0.5 * np.pi < x % (4 + np.pi) < 1.75 + 0.5 * np.pi  else 0) +
                     (2 if 2.25 + 0.5 * np.pi < x % (4 + np.pi) < 2.75 + 0.5 * np.pi  else 0) +  # événement ponctuel
                     0.1 * np.sin(2 * np.pi * 0.3 * e)  # irrégularité de l'environnement
                     )
```

Annexes

Annexe 3 : Programme python simulation boucle fermée avec PID

```python
# Conversion des listes en tableaux NumPy pour les calculs ultérieurs des critères de performance
sortie_r = np.array(sortie_r)
Cr = np.array(Cr)
t = np.array(t)

# Erreur totale robot
erreur_r = Cr - sortie_r

# Critères d'évaluation :
ISE = np.trapezoid(erreur_r**2, t)          # Integral of Squared Error
IAE = np.trapezoid(np.abs(erreur_r), t)     # Integral of Absolute Error
ITSE = np.trapezoid(t * erreur_r**2, t)     # Integral of Time * Squared Error
ITAE = np.trapezoid(t * np.abs(erreur_r), t) # Integral of Time * Absolute Error

print("\n=== Évaluation des performances PID avec Kalman ===")
print(f"IAE   (∫|e|dt)     = {IAE:.4f}")
print(f"ISE   (∫e²dt)      = {ISE:.4f}")
print(f"ITAE  (∫t|e|dt)    = {ITAE:.4f}")
print(f"ITSE  (∫t·e²dt)    = {ITSE:.4f}")

# Affichage des résultats
plt.figure(figsize=(20, 3))
plt.subplot(2, 2, 1)
plt.plot(t, sortie_g, label="Réponse")
plt.plot(t, Cg, color='r', linestyle='--', label="Consigne" )
plt.xlabel("Temps (s)")
plt.ylabel("Vitesse angulaire (rad/s)")
plt.title("Moteur gauche")
plt.grid(True)
plt.legend()

plt.subplot(2, 2, 2)
plt.title("Moteur droit", loc='center')
plt.plot(t, sortie_d, label="Réponse")
plt.axhline(y=12.56, color='r', linestyle='--', label="Consigne")
plt.xlabel("Temps (s)")
plt.ylabel("Vitesse angulaire (rad/s)")
plt.grid(True)
plt.legend()

plt.subplot(2, 1, 2)
plt.title("Robot", loc='center')
plt.plot(t, sortie_r, label="Réponse")
plt.plot(t, Cr, color='r', linestyle='--', label="Consigne")
plt.xlabel("Temps (s)")
plt.ylabel("Vitesse angulaire (rad/s)")
plt.grid(True)
plt.legend()

plt.subplots_adjust(top=0.9)
plt.suptitle("Figure 14 : Boucle fermée - Réponse des moteurs et du robot avec régulation PID sans bruit (2, 6, 0.005116)", fontsize=16)
plt.show()
```

Annexes

Annexe 4 : Programme python simulation boucle fermée avec PID avec filtre Kalman 1d

```python
import numpy as np
import matplotlib.pyplot as plt

# Paramètres du simulateur et du PID
K_g, K_d = 2.3, 2.3
tau_g, tau_d = 0.15, 0.25
dt = 0.01
t_max = 50
# Aucun code relatif à l'animation du robot n'est ajouté.
Kp_g, Ki_g, Kd_g = 2, 6, 0.005116
Kp_d, Ki_d, Kd_d = 2, 6, 0.005116

consigne_g = consigne_d = 12.56637061
perturbations = 0

t = np.arange(0, t_max, dt)
reponse_g = reponse_d = 0

# Initialisation des listes pour stocker les valeurs
sortie_g, sortie_d, sortie_r = [], [], []
mesures_g, mesures_d = [], []
kalman_g, kalman_d = [], []
X = []
Cg, Cr = [], []
x = 0

integrale_g = integrale_d = 0
erreur_precedente_g = erreur_precedente_d = 0

# Paramètres du filtre de Kalman pour chaque moteur
Q = 0.1  # bruit du modèle
R = 0.01  # bruit de mesure

# Initialisation des estimations et covariances au démarrage
x_hat_g, P_g = 0, 1.0
x_hat_d, P_d = 0, 1.0

def kalman_update(x_hat, P, mesure, Q, R):
    P = P + Q
    K = P / (P + R)
    x_hat = x_hat + K * (mesure - x_hat)
    P = (1 - K) * P
    return x_hat, P

for e in t:
    Cg.append(consigne_g)
    Cr.append(0.5 * (consigne_g + consigne_d))

    # Simulation d'une mesure bruitée pour chaque moteur
    mesure_g = reponse_g + np.random.normal(0, np.sqrt(R))
    mesure_d = reponse_d + np.random.normal(0, np.sqrt(R))
    mesures_g.append(mesure_g)
    mesures_d.append(mesure_d)
```

```python
    # Mise à jour du filtre de Kalman pour chaque moteur
    x_hat_g, P_g = kalman_update(x_hat_g, P_g, mesure_g, Q, R)
    x_hat_d, P_d = kalman_update(x_hat_d, P_d, mesure_d, Q, R)
    kalman_g.append(x_hat_g)
    kalman_d.append(x_hat_d)

    # Utilisation des estimations filtrées pour le calcul de l'erreur du PID
    erreur_g = consigne_g - x_hat_g
    erreur_d = consigne_d - x_hat_d
    integrale_g += erreur_g * dt
    integrale_d += erreur_d * dt
    integrale_g = np.clip(integrale_g, -10, 10)
    integrale_d = np.clip(integrale_d, -10, 10)

    derivee_g = (erreur_g - erreur_precedente_g) / dt
    derivee_d = (erreur_d - erreur_precedente_d) / dt
    erreur_precedente_g = erreur_g
    erreur_precedente_d = erreur_d

    commande_g = Kp_g * erreur_g + Ki_g * integrale_g + Kd_g * derivee_g
    commande_d = Kp_d * erreur_d + Ki_d * integrale_d + Kd_d * derivee_d
    commande_g = np.clip(commande_g, -20, 20)
    commande_d = np.clip(commande_d, -20, 20)

    # Simulation de la réponse du moteur
    reponse_g += (K_g * commande_g + perturbations - reponse_g) * (dt / tau_g)
    reponse_d += (K_d * commande_d + perturbations - reponse_d) * (dt / tau_d)

    # Calcul des vitesses et position
    vg = 0.03 * reponse_g
    vd = 0.03 * reponse_d
    vr = 0.5 * (vg + vd)
    x += vr * dt

    X.append(x)
    sortie_g.append(reponse_g)
    sortie_d.append(reponse_d)
    sortie_r.append(0.5 * (reponse_g + reponse_d))

    # Mise à jour de la consigne en fonction de la position du robot
    if x % (4 + np.pi) > 1:
        consigne_g = 8.11
    if x % (4 + np.pi) > (1 + 0.5 * np.pi):
        consigne_g = 12.56
    if x % (4 + np.pi) > (3 + 0.5 * np.pi):
        consigne_g = 8.11
    if x % (4 + np.pi) > (3 + np.pi):
        consigne_g = 12.56

    # Ajout de perturbations aléatoires et événements ponctuels
    perturbations = (0.2 * np.random.normal() +
                     (-2 if 1.25 + 0.5 * np.pi < x % (4 + np.pi) < 1.75 + 0.5 * np.pi else 0) +
                     (2 if 2.25 + 0.5 * np.pi < x % (4 + np.pi) < 2.75 + 0.5 * np.pi else 0) +
                     0.1 * np.sin(2 * np.pi * 0.3 * e)
                     )
```

Annexes

Annexe 4 : Programme python simulation boucle fermée avec PID avec filtre Kalman 1d

```
# Calcul des critères de performance
sortie_r = np.array(sortie_r)
Cr = np.array(Cr)
t = np.array(t)
erreur_r = Cr - sortie_r

ISE = np.trapz(erreur_r**2, t)
IAE = np.trapz(np.abs(erreur_r), t)
ITSE = np.trapz(t * erreur_r**2, t)
ITAE = np.trapz(t * np.abs(erreur_r), t)

print("\n=== Évaluation des performances PID avec Kalman ===")
print(f"IAE   (∫|e|dt)     = {IAE:.4f}")
print(f"ISE   (∫e²dt)      = {ISE:.4f}")
print(f"ITAE  (∫t|e|dt)    = {ITAE:.4f}")
print(f"ITSE  (∫t·e²dt)    = {ITSE:.4f}")

# Affichage des résultats
plt.figure(figsize=(18, 8))

plt.subplot(2, 2, 1)
plt.plot(t, sortie_g, label="Réponse réelle moteur gauche")
plt.plot(t, mesures_g, label="Mesure bruitée gauche", alpha=0.5)
plt.plot(t, kalman_g, label="Kalman gauche", linestyle='--')
plt.plot(t, Cg, color='r', linestyle='--', label="Consigne")
plt.xlabel("Temps (s)")
plt.ylabel("Vitesse angulaire (rad/s)")
plt.title("Moteur gauche")
plt.grid(True)
plt.legend()

plt.subplot(2, 2, 2)
plt.plot(t, sortie_d, label="Réponse réelle moteur droit")
plt.plot(t, mesures_d, label="Mesure bruitée droit", alpha=0.5)
plt.plot(t, kalman_d, label="Kalman droit", linestyle='--')
plt.axhline(y=12.56, color='r', linestyle='--', label="Consigne")
plt.xlabel("Temps (s)")
plt.ylabel("Vitesse angulaire (rad/s)")
plt.title("Moteur droit")
plt.grid(True)
plt.legend()

plt.subplot(2, 1, 2)
plt.plot(t, sortie_r, label="Réponse du robot")
plt.plot(t, Cr, color='r', linestyle='--', label="Consigne")
plt.xlabel("Temps (s)")
plt.ylabel("Vitesse (m/s)")
plt.title("Réponse du robot")
plt.grid(True)
plt.legend()

plt.suptitle("Figure 15 : Boucle fermée - Réponse des moteurs et du robot avec régulation PID avec filtre Kalman 1d (2, 6, 0.005116)", fontsize=16)
plt.show()
```

Annexes

Annexe 5 : Programme python fonction simulation boucle fermée avec PID pour générations et tests de coefficients de PID

```python
import numpy as np

def simuler_pid(Kp_g, Ki_g, Kd_g, Kp_d, Ki_d, Kd_d):
    # Configuration moteur
    K_g, tau_g = 2.3, 0.15
    K_d, tau_d = 2.3, 0.25
    dt = 0.01     # pas de temps
    t_max = 50   # durée de la simulation

    consigne_base=12.56
    reponse_g, reponse_d, x = 0, 0, 0
    sortie_r, Cr = [], []

    integrale_g, integrale_d = 0, 0
    erreur_precedente_g, erreur_precedente_d = 0, 0

    consigne_g = consigne_d = consigne_base

    t = np.arange(0, t_max, dt)

    for e in t:
        if x % (4 + np.pi) > 1:
            consigne_g = 8.11
        if x % (4 + np.pi) > (1 + 0.5 * np.pi):
            consigne_g = 12.56
        if x % (4 + np.pi) > (3 + 0.5 * np.pi):
            consigne_g = 8.11
        if x % (4 + np.pi) > (3 + np.pi):
            consigne_g = 12.56

        Cr.append(0.5 * (consigne_g + consigne_d))

        erreur_g = consigne_g - reponse_g
        erreur_d = consigne_d - reponse_d

        integrale_g += erreur_g * dt
        integrale_d += erreur_d * dt
```

```python
        derivee_g = (erreur_g - erreur_precedente_g) / dt
        derivee_d = (erreur_d - erreur_precedente_d) / dt

        erreur_precedente_g = erreur_g
        erreur_precedente_d = erreur_d

        commande_g = Kp_g * erreur_g + Ki_g * integrale_g + Kd_g * derivee_g
        commande_d = Kp_d * erreur_d + Ki_d * integrale_d + Kd_d * derivee_d

        commande_g = np.clip(commande_g, -20, 20)
        commande_d = np.clip(commande_d, -20, 20)

        perturbations = (
            0.2 * np.random.normal() +
            (-2 if 1.25 + 0.5 * np.pi < x % (4 + np.pi) < 1.75 + 0.5 * np.pi else 0) +
            (2 if 2.25 + 0.5 * np.pi < x % (4 + np.pi) < 2.75 + 0.5 * np.pi else 0) +
            0.1 * np.sin(2 * np.pi * 0.3 * e)
        )

        reponse_g += (K_g * commande_g + perturbations - reponse_g) * (dt / tau_g)
        reponse_d += (K_d * commande_d + perturbations - reponse_d) * (dt / tau_d)

        vr = 0.03 * (reponse_g + reponse_d) / 2
        x += vr * dt
        sortie_r.append((reponse_g + reponse_d) / 2)

    sortie_r = np.array(sortie_r)
    Cr = np.array(Cr)
    erreur_r = Cr - sortie_r

    itae = np.trapezp(t * np.abs(erreur_r), t)
    return itae, sortie_r, Cr
```

Annexes

Annexe 6 : Programme python générations et tests de coefficients de PID

```python
import sanstitre2
import numpy as np
import time
import torch

N = 100 # Nombre d'essais
# Initialisation des variables pour conserver le meilleur score et les paramètres PID correspondants
meilleur_score = float("inf")
meilleur_PID = None
meilleure_sortie = None
meilleure_consigne = None  # Variable définie mais non utilisée dans le code actuel

# Début du chronométrage de l'exécution
c = time.time()

# Boucle d'exécution pour N essais
for _ in range(N):
    # Génération aléatoire de coefficients PID pour gauche et droite
    Kp_g = np.random.uniform(7, 10)
    Kp_d = Kp_g  # Assure que le coefficient Kp est identique pour les deux côtés
    Ki_g = np.random.uniform(1, 3)
    Ki_d = Ki_g  # Même logique pour Ki
    Kd_g = np.random.uniform(0.03, 0.04)
    Kd_d = Kd_g  # et pour Kd

    # Simulation PID avec les coefficients générés
    score_itae, sortie_r, Cr = sanstitre2.simuler_pid(Kp_g, Ki_g, Kd_g, Kp_d, Ki_d, Kd_d)

    # Calcul de l'erreur entre la consigne et la sortie réelle
    erreur_r = Cr - sortie_r
    # Calcul du dépassement
    depassement = np.max(sortie_r) - np.max(Cr)
    # Calcul de l'oscillation sur les 20% finaux de la trajectoire
    oscillation = np.std(erreur_r[-int(len(erreur_r) * 0.2):])

    # Calcul d'un score total combinant score ITAE, dépassement et oscillation
    score_total = score_itae + 10 * depassement + 20 * oscillation

    # Si le système est trop instable (dépassement ou oscillation trop élevés), on rejette la solution
    if depassement > 1 or oscillation > 1:
        continue

    # Affichage du score total pour la configuration actuelle
    print(score_total)

    # Mise à jour du meilleur score et des paramètres PID si le score total actuel est inférieur
    if score_total < meilleur_score:
        meilleur_score = score_total
        meilleur_PID = (Kp_g, Ki_g, Kd_g, Kp_d, Ki_d, Kd_d)
        meilleure_sortie = sortie_r

    # Enregistrement progressif des meilleurs PID trouvés dans un fichier texte
    with open('C:/Users/ferna/OneDrive/Bureau/TIPE/simulation/V2/scores.txt', "a") as s:
        s.write("\n" + str(meilleur_PID[0]) + " " + str(meilleur_PID[1]) + " " + str(meilleur_PID[2]) + "\t" + str(meilleur_score))

# Affichage du meilleur PID trouvé et de son score après toutes les tentatives
print("\n=== Meilleur PID trouvé ===")
print("Kp_g, Ki_g, Kd_g = " + str(meilleur_PID[0]) + str(meilleur_PID[1]) + str(meilleur_PID[2]))
print("\nScore = " + str(meilleur_score))

# Affichage du temps total d'exécution de la simulation
print(time.time() - c)
```

Annexes

Annexe 7 : Programme python générations et tests de coefficients de PID en parallèle

```python
import numpy as np
import concurrent.futures
import heapq
import os
import sanstitre2  # Ce fichier doit contenir uniquement des fonctions

def tester_combinaison(_):
    # Crée un générateur local par appel (ou pense à le créer une seule fois par processus via un setup)
    rng = np.random.default_rng()

    # Génération aléatoire des coefficients PID
    Kp = rng.uniform(2, 5)    # Intervalle pour Kp
    Ki = rng.uniform(5, 20)   # Intervalle pour Ki
    Kd = rng.uniform(0.001, 0.2) # Intervalle pour Kd

    # Appel de la simulation avec les coefficients PID générés et récupération des résultats
    score_itae, sortie_r, Cr = sanstitre2.simuler_pid(Kp, Ki, Kd, Kp, Ki, Kd)
    # Calcul de l'erreur entre la sortie prévue et la référence
    erreur_r = Cr - sortie_r

    # Calcul du dépassement et de l'oscillation (écart-type sur la dernière portion du signal)
    depassement = np.max(sortie_r) - np.max(Cr)
    oscillation = np.std(erreur_r[-int(len(erreur_r)*0.2):])

    # Score total combinant l'ITAE et les pénalités pour le dépassement et l'oscillation
    score_total = score_itae + 10 * depassement + 20 * oscillation

    # Filtrer les résultats qui dépassent les seuils fixés
    if depassement > 1 or oscillation > 1:
        return None
    return (score_total, Kp, Ki, Kd)

if __name__ == "__main__":
    # Nombre total d'essais
    N = 1000
    # Nombre de meilleurs résultats à conserver
    TOP_N = 1000
    start = time.time()
    print("Lancement")

    # Détermination du nombre maximum de processus à utiliser
    max_workers = min(8, os.cpu_count() or 1)
    # Calcul d'une taille de chunk adaptée pour optimiser le traitement en parallèle
    chunk_size = max(N // (max_workers * 4), 1)

    # Utilisation de ProcessPoolExecutor pour lancer en parallèle les simulations
    with concurrent.futures.ProcessPoolExecutor(max_workers=max_workers) as executor:
        resultats = list(executor.map(tester_combinaison, range(N), chunksize=chunk_size))

    # Filtrer les résultats non valides (ceux pour lesquels la fonction retourne None)
    resultats = [r for r in resultats if r is not None]

    # Sélectionner les TOP_N meilleurs résultats en fonction du score total (le plus petit est le meilleur)
    resultats_tries = heapq.nsmallest(TOP_N, resultats, key=lambda x: x[0])

    # Chemin d'accès au fichier de scores
    chemin_scores = os.path.join('C:/Users/ferna/OneDrive/Bureau/TIPE/simulation/V2', 'scores.txt')

    # Écriture des scores et des valeurs PID dans le fichier scores.txt
    with open(chemin_scores, "a") as f:
        for score, kp, ki, kd in resultats_tries:
            f.write(f"{kp:.6f}\t{ki:.6f}\t{kd:.6f}\t{score:.6f}\n")

    # Affichage des résultats +: meilleur score et ses coefficients PID correspondants
    if resultats_tries:
        best = resultats_tries[0]
        print(f"\nTerminé en {time.time() - start:.2f}s")
        print(f"Meilleur PID : Kp = {best[1]:.3f}, Ki = {best[2]:.3f}, Kd = {best[3]:.3f}")
        print(f"Score total : {best[0]:.3f}")
    else:
        print("Aucune combinaison PID valide trouvée.")
```

Annexes

Annexe 8 : Programme python pour trier par ordre croissant les scores

```python
import matplotlib.pyplot as plt

def lire_et_trier_scores(fichier_entree, fichier_sortie):
    # Initialisation d'une liste vide pour stocker les tuples (score, ligne)
    lignes = []

    # Lecture du fichier d'entrée
    with open(fichier_entree, "r") as f:
        # Lecture de toutes les lignes du fichier
        lignes_brutes = f.readlines()

    # Extraction des deux premières lignes comme en-tête
    en_tete = lignes_brutes[:2]
    # Les lignes suivantes contiennent les données
    lignes_donnees = lignes_brutes[2:]

    # Traiter chaque ligne de données
    for ligne in lignes_donnees:
        # Suppression des espaces en début et fin et séparation par espaces
        elements = ligne.strip().split()
        # Vérification que la ligne contient exactement 4 éléments
        if len(elements) == 4:
            kp, ki, kd, score = elements
            try:
                # Conversion de l'élément score en float pour pouvoir trier
                score = float(score)
                # Ajout d'un tuple (score, ligne) à la liste
                lignes.append((score, ligne.strip()))
            except ValueError:
                # En cas d'erreur de conversion, on ignore simplement la ligne mal formatée
                continue

    # Tri de la liste de tuples en fonction du score (valeur float) en ordre croissant
    lignes_tries = sorted(lignes, key=lambda x: x[0])

    # Ecriture des données triées dans le fichier de sortie
    with open(fichier_sortie, "w") as f:
        # Ecriture de l'en-tête dans le fichier de sortie
        f.writelines(en_tete)
        # Ecriture de chaque ligne triée avec un retour à la fin de ligne
        for _, ligne in lignes_tries:
            f.write(ligne + "\n")

    # Affichage d'un message indiquant la réussite de l'opération
    print(f"Scores triés par ordre croissant écrits dans {fichier_sortie}")
```

Annexes

Annexe 9 : Programme python pour lire les mesures réelles sur un fichier

```python
from datetime import datetime
import matplotlib.pyplot as plt

def lire_donnees(nom_fichier):
    temps = []
    sortie_g = [] # Réponse du moteur gauche
    sortie_d = [] # Réponse du moteur droit
    # Ouvre le fichier en mode lecture
    with open(nom_fichier, 'r') as fichier:
        # Parcours chaque ligne du fichier
        for ligne in fichier:
            # Supprime espaces en début/fin et remplace les virgules par rien
            ligne_nettoyee = ligne.strip().replace(',', '')
            # Passe à la ligne suivante si la ligne est vide
            if not ligne_nettoyee:
                continue
            # Sépare la ligne en éléments individuels
            parties = ligne_nettoyee.split()
            # Continue seulement si au moins trois éléments (temps, moteur gauche, moteur droit) sont présents
            if len(parties) < 3:
                continue
            # Extrait la chaîne de temps et les valeurs des moteurs gauche et droit
            temps_str, g, d = parties[:3]
            try:
                # Convertit la chaîne de temps en un objet datetime
                t = datetime.strptime(temps_str, "%H:%M:%S.%f")
                # Ajoute le temps converti à la liste des temps
                temps.append(t)
                # Convertit et ajoute la valeur du moteur gauche
                sortie_g.append(float(g))
                # Convertit et ajoute la valeur du moteur droit
                sortie_d.append(float(d))
            except Exception:
                # En cas d'erreur lors de la conversion, ignore la ligne
                continue
    # Retourne les listes de temps et des sorties des moteurs gauche et droit
    return temps, sortie_g, sortie_d
```

```python
def main():
    nom_fichier = "C:/Users/ferna/OneDrive/Bureau/TIPE/simulation/V2/mes_ouv.txt"
    temps, sortie_g, sortie_d = lire_donnees(nom_fichier)
    if not temps:
        print("Aucune donnée valide trouvée.")
        return

    # Conversion en secondes depuis le premier relevé
    temps_debut = temps[0]
    temps_sec = [(t - temps_debut).total_seconds() for t in temps]

    # Calcul de la vitesse angulaire du robot (moyenne des deux moteurs)
    sortie_r = [(g + d) / 2 for g, d in zip(sortie_g, sortie_d)]

    plt.figure(figsize=(20, 3))
    plt.suptitle("Figure 16 : Boucle ouverte - Réponse réelle des moteurs et du robot sans régulation")

    plt.subplot(2, 2, 1)
    plt.plot(temps_sec, sortie_g)
    plt.title("Moteur gauche", loc='center')
    plt.axhline(y=12.56, color='r', linestyle='--')
    plt.xlabel("Temps (s)")
    plt.ylabel("Vitesse angulaire (rad/s)")
    plt.grid(True)

    plt.subplot(2, 2, 2)
    plt.plot(temps_sec, sortie_d)
    plt.title("Moteur droit", loc='center')
    plt.axhline(y=12.56, color='r', linestyle='--')
    plt.xlabel("Temps (s)")
    plt.ylabel("Vitesse angulaire (rad/s)")
    plt.grid(True)

    plt.subplot(2, 1, 2)
    plt.plot(temps_sec, sortie_r)
    plt.axhline(y=12.56, color='r', linestyle='--')
    plt.title("Robot", loc='center')
    plt.xlabel("Temps (s)")
    plt.ylabel("Vitesse angulaire (rad/s)")
    plt.grid(True)
    plt.show()

if __name__ == '__main__':
    main()
```

Annexes

Annexe 10 : Programme Arduino pour la motorisation en boucle ouverte

```
#include "SoftwareSerial.h"
SoftwareSerial mySerial(10, 11); // Création d'un objet pour la communication sur les broches 10 et 11
// Définition des broches utilisées pour le contrôle des moteurs
#define E1 5    // Broche de contrôle (PWM) pour la vitesse du moteur gauche
#define M1 4    // Broche de direction pour le moteur gauche
#define E2 6    // Broche de contrôle (PWM) pour la vitesse du moteur droit
#define M2 2    // Broche de direction pour le moteur droit
void setup() {
  // Initialisation de la communication série pour le SoftwareSerial et le port série principal
  mySerial.begin(9600);
  Serial.begin(9600);

  // Définition des broches des moteurs en mode sortie
  pinMode(M1, OUTPUT);
  pinMode(M2, OUTPUT);
  pinMode(E1, OUTPUT);
  pinMode(E2, OUTPUT);
}
void loop() {
  // Vérification des données disponibles sur la liaison série logicielle
  if (mySerial.available()){
    // Lecture de la commande reçue jusqu'au caractère '\n' de passage à la ligne
    String cmd = mySerial.readStringUntil('\n');
    cmd.trim(); // Supprime les espaces superflus en début et fin de chaîne
    // Exemple de traitement des commandes reçues:
    if (cmd == "A") {
      avancer(HIGH, HIGH, 155, 155); // Faire avancer les moteurs à une vitesse standard
    }
    else if (cmd == "G") {
      avancer(HIGH, HIGH, 100, 155); // Tourner à gauche: réduire la vitesse du moteur gauche}
    else if (cmd == "D") {
      avancer(HIGH, HIGH, 155, 100); // Tourner à droite: réduire la vitesse du moteur droit}
    else {
      avancer(LOW, LOW, 0, 0); // Arrêter les moteurs si la commande est non reconnue
    }
  }
}

// Fonction pour contrôler l'avancée des moteurs
void avancer(char G, char D, int vg, int vd) {
  digitalWrite(M1, G);  // Appliquer l'état de direction sur le moteur gauche
  digitalWrite(M2, D);  // Appliquer l'état de direction sur le moteur droit
  analogWrite(E1, vg);  // Régler la vitesse du moteur gauche
  analogWrite(E2, vd);  // Régler la vitesse du moteur droit
}
```

Annexes

Annexe 11 : Programme Arduino pour les capteurs en boucle ouverte

```
#include "SoftwareSerial.h"
//#include "Ultrasonic.h"

// Déclaration des ports série logiciels pour communication
SoftwareSerial mySerial(11,10);
SoftwareSerial mySerialBT(12,13); // Bluetooth
//Ultrasonic ultrasonic(13);
//int distance;

// Définition des broches pour les capteurs et boutons
#define BP A0
#define Enc1 2
#define Enc2 3

// Variables pour les capteurs de ligne
bool CAPG, CAPC, CAPD;

// Variables pour compter les impulsions des encodeurs
volatile int pulseCount1 = 0;
volatile int pulseCount2 = 0;
unsigned long lastTime = 0;
int omega1, omega2;

// Variables pour calcul des moyennes et statistiques
int e1=0,e2=0,e3=0,sAg=0,sGg=0,sDg=0,sAd=0,sGd=0,sDd=0,mAg=0,mGg=0,mDg=0,mAd=0,mGd=0,mDd=0;
bool affmoy=false;
bool actif = false;

// Variables pour la gestion des commandes
String currentCmd;
String lastCmd = "";

// Initialisation des ports et interruptions
void setup() {
  mySerial.begin(9600);
  mySerialBT.begin(9600);
  Serial.begin(9600);
  pinMode(4, INPUT);    // Capteur gauche
  pinMode(5, INPUT);    // Capteur central
  pinMode(6, INPUT);    // Capteur droit
  pinMode(BP, INPUT);   // Bouton poussoir
  pinMode(Enc1, INPUT); // Encodeur 1
  pinMode(Enc2, INPUT); // Encodeur 2
  attachInterrupt(digitalPinToInterrupt(Enc1), countPulses1, RISING); // Interruption sur encodeur 1
  attachInterrupt(digitalPinToInterrupt(Enc2), countPulses2, RISING); // Interruption sur encodeur 2
```

```
// Boucle principale
void loop() {
  app();
}

// Fonction d'interruption pour l'encodeur 1
void countPulses1() {
  pulseCount1++;
}

// Fonction d'interruption pour l'encodeur 2
void countPulses2() {
  pulseCount2++;
}

// Calcul de la vitesse des roues à intervalles réguliers
void vitesse() {
  if (millis() - lastTime >= 100) { // toutes les 100 ms
    omega1 = (pulseCount1 / 20.0) * 20.0 * 3.14159; // Calcul vitesse roue 1
    omega2 = (pulseCount2 / 20.0) * 20.0 * 3.14159; // Calcul vitesse roue 2
    mySerialBT.print(omega1);
    mySerialBT.print(" , ");
    mySerialBT.println(omega2);
    pulseCount1 = 0;
    pulseCount2 = 0;
    lastTime = millis();
    moyenne(); // Mise à jour des moyennes
  }
}

// Calcul des moyennes pour chaque commande
void moyenne() {
  if (omega1!=0 || omega2!=0 || currentCmd=="S") {
    if (currentCmd=="A"){ // Avancer
      e1++;
      sAd+=omega1;
      sAg+=omega2;
      affmoy=false;
    }
    else if (currentCmd=="G"){ // Tourner à gauche
      e2++;
      sGd+=omega1;
      sGg+=omega2;
      affmoy=false;
    }
```

Annexes

Annexe 11 : Programme Arduino pour les capteurs en boucle ouverte

```
    else if (currentCmd=="D"){ // Tourner à droite
      e3++;
      sDd+=omega1;
      sDg+=omega2;
      affmoy=false;
    }
    else if (!affmoy && (e1!=0 && e2!=0 && e3!=0)) { // Calcul des moyennes si toutes les directions ont été testées
      mAg=sAg/e1;
      mAd=sAd/e1;
      mGg=sGg/e2;
      mGd=sGd/e2;
      mDg=sDg/e3;
      mDd=sDd/e3;
      affmoy=true;
    }
  }
}

// Lecture des capteurs de ligne et détermination de la commande à envoyer
void line() {
  CAPG = digitalRead(4); // Capteur gauche
  CAPC = digitalRead(5); // Capteur central
  CAPD = digitalRead(6); // Capteur droit

  if (CAPC) {
    currentCmd="A"; // Avancer
  }
  else if (CAPG) {
    currentCmd="G"; // Tourner à gauche
  }
  else if (CAPD) {
    currentCmd="D"; // Tourner à droite
  }
  else {
    //ult();
    currentCmd="S"; // Stop si aucun capteur n'est activé
  }

  // Envoi de la commande si elle a changé
  if (currentCmd != lastCmd) {
    mySerial.println(currentCmd);
    Serial.println(currentCmd);
```

```
    lastCmd = currentCmd;
  }
}

// Fonction principale de gestion de l'application
void app() {
  // Lecture des commandes reçues par Bluetooth
  if (mySerialBT.available()) {
    String val = mySerialBT.readStringUntil('\n');
    val.trim(); // Supprime les espaces superflus
    if (val == "1") {
      actif = true; // Activation du suivi de ligne
    }
    else if (val == "0") {
      actif = false; // Désactivation
      mySerial.println("S");
    }
    else if (val == "moyenne") { // Demande d'affichage des moyennes
      mySerialBT.print(mAg);
      mySerialBT.print(",");
      mySerialBT.print(mAd);
      mySerialBT.print(",");
      mySerialBT.print(mGg);
      mySerialBT.print(",");
      mySerialBT.print(mGd);
      mySerialBT.print(",");
      mySerialBT.print(mDg);
      mySerialBT.print(",");
      mySerialBT.println(mDd);
      affmoy = false;
    }
  }

  // Arrêt d'urgence via bouton poussoir
  if (digitalRead(BP) == HIGH) {
    actif = false;
    mySerial.println("S");
  }

  // Si le robot est actif, suivi de ligne et calcul de vitesse
  if (actif) {
    line();
    vitesse();
```

Annexes

Annexe 12 : Programme Arduino pour la motorisation en boucle fermée avec/sans PID

```
#include "SoftwareSerial.h"
SoftwareSerial mySerial(10,11); // RX, TX
// Définition des broches pour les moteurs
#define E1 5   // PWM moteur gauche
#define M1 4   // Direction moteur gauche
#define E2 6   // PWM moteur droit
#define M2 2   // Direction moteur droit
void setup() {
  // Initialisation de la communication série pour le SoftwareSerial et le port série principal
  mySerial.begin(9600);
  Serial.begin(9600);

  // Définition des broches des moteurs en mode sortie
  pinMode(M1, OUTPUT);
  pinMode(M2, OUTPUT);
  pinMode(E1, OUTPUT);
  pinMode(E2, OUTPUT);
}
void loop() {
  // Vérifie si des données sont disponibles sur le port série logiciel
  if (mySerial.available()){
    String ins = mySerial.readStringUntil('\n'); // Lit la ligne reçue jusqu'au saut de ligne
    ins.trim(); // Supprime les espaces inutiles
    //Serial.println(ins);
    int sep1 = ins.indexOf(':'); // Cherche le premier séparateur ':'
    int sep2 = ins.indexOf(':', sep1 + 1); // Cherche le second séparateur ':'

    // Vérifie que les deux séparateurs existent
    if (sep1 != -1 && sep2 != -1) {
      String cmd = ins.substring(0, sep1);   // Récupère la commande (A, G, D, etc.)
      cmd.trim();
      int Output1 = ins.substring(sep1 + 1, sep2).toInt(); // Récupère la valeur du moteur gauche
      int Output2 = ins.substring(sep2 + 1).toInt();       // Récupère la valeur du moteur droit

      Serial.println(cmd); // Affiche la commande reçue sur le port série

      // Si la commande est reconnue, fait avancer les moteurs
      if (cmd=="A" || cmd=="G" || cmd=="D") {
        avancer(HIGH,HIGH,Output1,Output2);
      }
      // Sinon, arrête les moteurs
      else {
        avancer(LOW,LOW,0,0);
      }
    }
  }
}

// Fonction pour contrôler les moteurs
void avancer(char G, char D, int vg, int vd) {
  digitalWrite(M1, G);  // Définit la direction du moteur gauche
  digitalWrite(M2, D);  // Définit la direction du moteur droit
  analogWrite(E1, vg);  // Définit la vitesse du moteur gauche (PWM)
  analogWrite(E2, vd);  // Définit la vitesse du moteur droit (PWM)
}
```

Annexes

Annexe 13 : Programme Arduino pour les capteurs en boucle fermée avec PID

```
#include "SoftwareSerial.h"
//#include "Ultrasonic.h"
SoftwareSerial mySerial(11,10);
SoftwareSerial mySerialBT(12,13); // Bluetooth
//Ultrasonic ultrasonic(13);
//int distance;
#include <PID_v1.h>

#define BP A0      // Bouton poussoir
#define Enc1 2     // Encodeur moteur 1
#define Enc2 3     // Encodeur moteur 2

// Variables pour capteurs de ligne
bool CAPG, CAPC, CAPD;

// Variables pour compter les impulsions des encodeurs
volatile int pulseCount1 = 0;
volatile int pulseCount2 = 0;
unsigned long lastTime = 0;
int omega1=0, omega2=0, omega155=12.56; // Vitesses moteurs

// Variables pour calcul de moyennes
int m1=0,m2=0,m3=0,mAg=0,mGg=0,mDg=0,mAd=0,mGd=0,mDd=0;
bool affmoy=false;
bool actif=false;

String Cmd; // Commande courante

// Variables PID
double Setpoint1=155, Input1, Output1, Setpoint2=155, Input2, Output2;
double Kp=2,Ki=6,Kd=0.005116;
PID myPID1(&Input1, &Output1, &Setpoint1, Kp, Ki, Kd, DIRECT);
PID myPID2(&Input2, &Output2, &Setpoint2, Kp, Ki, Kd, DIRECT);

void setup() {
  // Initialisation des communications série
  mySerial.begin(9600);
  mySerialBT.begin(9600);
  Serial.begin(9600);

  // Initialisation des broches pour les capteurs et encodeurs
  pinMode(4, INPUT);          // Capteur gauche
  pinMode(5, INPUT);          // Capteur central
  pinMode(6, INPUT);          // Capteur droit
  pinMode(BP, INPUT);         // Bouton poussoir
  pinMode(Enc1, INPUT);       // Encodeur 1
  pinMode(Enc2, INPUT);       // Encodeur 2
  attachInterrupt(digitalPinToInterrupt(Enc1), countPulses1, RISING); // Interruption encodeur 1
  attachInterrupt(digitalPinToInterrupt(Enc2), countPulses2, RISING); // Interruption encodeur 2
  myPID1.SetMode(AUTOMATIC); // PID automatique
  myPID2.SetMode(AUTOMATIC);
}
```

```
void loop() {
  app(); // Boucle principale
}

// Calcul des PWM à partir des vitesses mesurées et PID
void calculPWM() {
  Input1 = 155*omega1/omega155;
  Input2 = 155 *omega2/omega155;

  myPID1.Compute();
  myPID2.Compute();
  Output1 = constrain(Output1, 0, 200); // Limiter PWM
  Output2 = constrain(Output2, 0, 200);
}

// Compteur d'impulsions encodeur 1
void countPulses1() {
  pulseCount1++;
}

// Compteur d'impulsions encodeur 2
void countPulses2() {
  pulseCount2++;
}

// Calcul de la vitesse des moteurs à partir des impulsions
void vitesse() {
  if (millis() - lastTime >= 100) { // Toutes les 100 ms
    omega1 = (pulseCount1 / 20.0) * 20.0 * 3.14159; // Calcul vitesse moteur 1
    omega2 = (pulseCount2 / 20.0) * 20.0 * 3.14159; // Calcul vitesse moteur 2
    mySerialBT.print(omega1);
    mySerialBT.print(" , ");
    mySerialBT.println(omega2);
    pulseCount1 = 0;
    pulseCount2 = 0;
    lastTime = millis();
    moyenne(); // Calcul des moyennes
  }
}

// Calcul des moyennes des vitesses selon la commande
void moyenne() {
  if (omega1!=0 || omega2!=0 || Cmd=="S") {
    if (Cmd=="A"){
      m1++;
      mAd+=omega1;
      mAg+=omega2;
      affmoy=false;
    }
    else if (Cmd=="G"){
      m2++;
      mGd+=omega1;
      mGg+=omega2;
      affmoy=false;
    }
```

Annexes

Annexe 13 : Programme Arduino pour les capteurs en boucle fermée avec PID

```
    else if (Cmd=="D"){
      m3++;
      mDd+=omega1;
      mDg+=omega2;
      affmoy=false;
    }
    else if (!affmoy && (m1!=0 && m2!=0 && m3!=0)) {
      affmoy=true;
    }
  }
}

// Lecture des capteurs de ligne et choix de la commande
void line() {
  CAPG = digitalRead(4); // Capteur gauche
  CAPC = digitalRead(5); // Capteur central
  CAPD = digitalRead(6); // Capteur droit

  if (CAPC) {
    Setpoint1 = 155;
    Setpoint2 = 155;
    Cmd="A";
  }
  else if (CAPG) {
    Setpoint1 = 155;
    Setpoint2 = 155;
    Cmd="G";
  }
  else if (CAPD) {
    Setpoint1 = 155;
    Setpoint2 = 155;
    Cmd="D";
  }
  else {
    //ult();
    Cmd="S"; // Stop
  }

  // Affichage des commandes et PWM sur le port série
  mySerial.print(Cmd);
  mySerial.print(":");
  mySerial.print(Output1);
  mySerial.print(":");
  mySerial.println(Output2);
}

// Application principale : gestion Bluetooth, bouton, calculs et actions
void app() {
  if (mySerialBT.available()) { // Si message Bluetooth reçu
    String val = mySerialBT.readStringUntil('\n');
    val.trim();
    //Serial.println(val);
    if (val == "1") { // Activation
```

```
    if (val == "1") { // Activation
      actif = true;
    }
    else if (val == "0") { // Désactivation
      actif = false;
      mySerial.println("S");
    }
    else if (val == "moyenne") { // Demande des moyennes
      mySerialBT.print(mAg/m1);
      mySerialBT.print(",");
      mySerialBT.print(mAd/m1);
      mySerialBT.print(",");
      mySerialBT.print(mGg/m2);
      mySerialBT.print(",");
      mySerialBT.print(mGd/m2);
      mySerialBT.print(",");
      mySerialBT.print(mDg/m3);
      mySerialBT.print(",");
      mySerialBT.println(mDd/m3);
      affmoy = false;
    }
  }

  if (digitalRead(BP) == HIGH) { // Arrêt d'urgence par bouton
    actif = false;
    mySerial.println("S");
  }

  if (actif) { // Si actif, exécuter les fonctions principales
    calculPWM();
    line();
    vitesse();
  }
}

//void ult() {
//long distance;
//  distance = ultrasonic.read();
//  delay(50);
//  Serial.print(distance);
//  Serial.println("cm");
//  if(distance == 7){
//    avancer(HIGH,155,155);}
//  if(distance < 5) {
//    avancer(HIGH,155,100);}
//  if(distance > 9){
//    avancer(HIGH,100,155);}}
```